Surrogate models for EMT6/Ro radiotherapy protocols:
project changes, results against the original methods, and the 400k dataset

Ignacy Kozakiewicz

6 October 2026

Abstract

This note documents what changed in the `cancertreatment/surogate_models` project between July and September 2026, how the resulting models compare with the methods they started from, and how a second 200 000-protocol dataset was generated and combined with the original one. The inherited configuration reproduces at 4.52–4.59 MAE over four complete runs. The GRU and event-based models developed from it reached 4.29 MAE for a single run and 4.19 MAE for a 14-run prediction average. All of these beat the best model of the thesis (4.87 MAE), and the average matches the upstream repository's headline figure (4.2 MAE / 95.3 % correct). Doubling the training data with 200k Sobol-sampled protocols simulated on the Rysy cluster then lowered every single model by 0.11–0.23 MAE and brought a five-run average to 4.03 MAE / 95.9 % correct on the like-for-like uniform test protocols, the first result under 1 % MAPE.

1 Headline numbers

model	data	MAE	RMSE	MAPE	correct	MAE_{99}
Thesis best model (3×CNNAttLSTM)	200k	4.87	6.47	1.15 %	94.77 %	19
Upstream README headline	200k	4.2	–	0.99 %	95.3 %	15.78
Baseline `1.yml` (reproduced)	200k	4.555	5.98	1.08 %	94.97 %	17.5
`24_gru_xl`, seed 1	200k	4.457	5.83	1.055 %	95.07 %	17.2
`42_event_gru_addlog`, best seed	200k	4.289	5.61	1.017 %	95.29 %	16.4
Average of 14 GRU-family runs	200k	4.186	5.45	0.992 %	95.35 %	15.8
`42_event_gru_addlog`, best seed	400k	4.175	5.45	0.990 %	95.72 %	15.7
Average of 5 runs	**400k**	**4.033**	**5.25**	**0.957 %**	**95.87 %**	**14.9**

Table 1: Test-split metrics. All 200k rows are scored on the 200k test split (20 000 uniformly sampled protocols). The 400k rows are scored on the 20 013 uniformly sampled protocols of the 400k test split, which is the like-for-like subset (Section 6). "Correct" is the fraction of random protocol pairs whose order the model gets right; MAE_{99} is the 99th percentile of the absolute error.

2 Starting point and evaluation protocol

The task. One series is one five-day irradiation protocol for an EMT6/Ro tumour spheroid: up to 20 doses (0.5–10 Gy total, multiples of 0.125 Gy, at least 30 minutes apart, on a 10-minute grid within the first 72 000 six-second steps, i.e. five days). The simulation itself runs for 144 000 steps (ten days), and the target is the number of live tumour cells at the end of it, averaged over 100 stochastic replicates of the cellular automaton. A surrogate model that predicts this number replaces the GPU simulator inside protocol optimisation.

The inherited configuration. The project started from the `AWarno/CancerDLOptimization` code base and its thesis. Its best configuration, `configs/cnn_lstm_multi_loss/1.yml`, is a `MultiHeadTaskRegressor` of three `CNNAttLSTM` sub-networks trained with three L1 heads plus a margin ranking loss (weight 5, margin −0.04). Three figures are attached to it. The thesis reports

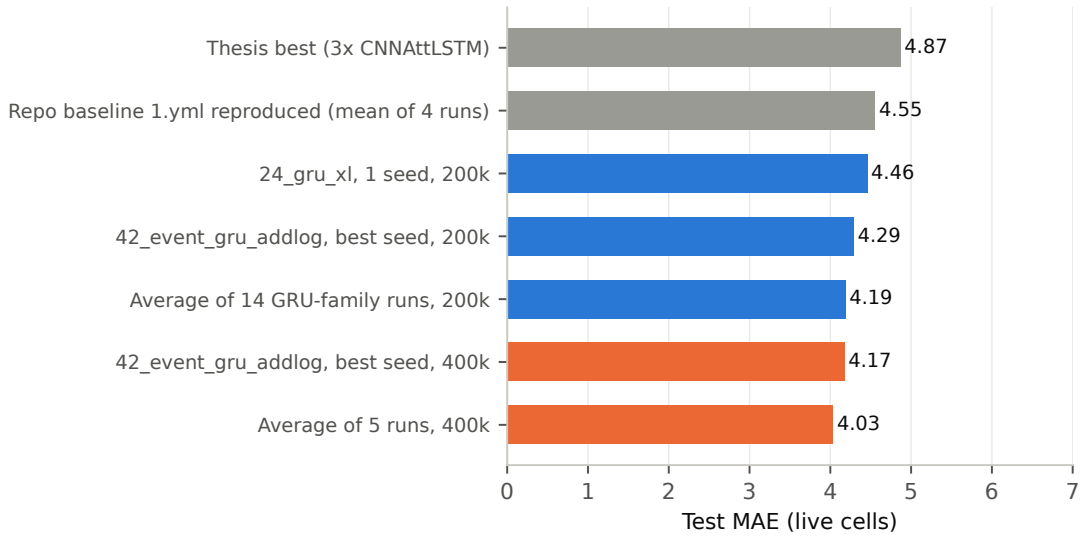


Figure 1: Test MAE of the reference methods (grey), the best 200k results of this project (blue) and the 400k results (orange).

1.15 % MAPE / 4.87 MAE / 94.77 % correct for its best model. The upstream README claims 0.99 % / 4.2 / 95.3 %. Four complete local runs of `1.yml` in July 2026 (170–200 epochs) score 4.52–4.59 MAE / 94.9–95.0 % correct, so the README figure is not entirely reproduced by `1.yml` itself.

Metrics. Every model is scored by one function, `cancer_nn.metrics.evaluate`, which takes the predictions for a whole split and returns five numbers in cells: MAE, RMSE, MAPE, correct-percent and MAE_{99}. Correct-percent pairs every series of the split with one other series through a fixed permutation (`random_state=0`) and counts the pairs whose predicted order matches the true order; MAE_{99} is the 99th percentile of the absolute error. The same function is called on the full validation split at the end of every epoch (logged as `val/*`), on the full test split once after training (`test/*`, the numbers quoted here), by `eval_ensemble.py` for every checkpoint, prediction average and bootstrap interval, and by the per-source breakdown of the 400k batch, so every entry in every table below comes from the same code path. During training only the losses are logged per batch. The thesis and the upstream README use the same pairing for correct-percent, so Table 1 is a direct comparison.

3 Changes to the models and training

3.1 Code changes

- **GRU family as the working baseline.** Plain `GRURegressor` configs 19–26 replaced the CNN-attention-LSTM as the working model from late July. The size sweep (one seed, 200k) went single-layer 7.46, tiny 7.95, medium 5.81, bidirectional 7.01, large 4.63, xl 4.457, xxl 4.624 and a diverse three-member ensemble 4.463 MAE, so `24_gru_xl` (392k parameters, the same count as `1.yml`) became the reference single model. It trains about 2.5× faster than the LSTM ensembles. A second seed of configs 23–26 (`summarize_gru_seed_sweep.py`, Welch's t-test) showed large, xl, xxl and the ensemble to be statistically indistinguishable ($p > 0.1$).
- **CNN-attention wrapper on the GRU** (`CNNAttGRU`, configs 27–29, August, one seed): medium 5.715, large 4.570, xl 4.565 MAE (775k parameters for xl). It helps at medium and large size but loses to the plain GRU at xl (4.565 vs 4.457), so the wrapper was not carried forward.
- **Optimiser, scheduler and SWA knobs**: `optimizer` (`adam`/`adamw` with `weight_decay`), `scheduler` (`plateau`/`onecycle`/`cosine`) and a `-swa_lrs` flag. Stacking Huber loss + AdamW +

OneCycle + SWA on the baseline LSTM ensemble (config 37: 4.885 MAE) and on `24_gru_xl` (config 38: 4.856 MAE) made both *worse* than their plain counterparts (4.55 and 4.46).

- **Event-based recurrence** (`event_gru`, `EventRecurrentRegressor`, 99k parameters): the hidden state changes only at real doses; a gap-conditioned residual MLP evolves it between doses, a `GRUCell` applies each dose, and the state is evolved to the end of the 72 000-step dosing window before readout. Padding rows are masked.
- **Additive log readout** (`readout: additive_log`): the prediction is $\exp\left(\log y_0 + \sum_i c(h_i) + c_{\text{end}}(h_T)\right)$, i.e. leftover cells as a product of per-dose factors and an end-of-treatment factor, with contribution heads initialised at zero so training starts from the mean target. Combined with event recurrence this is config `42_event_gru_addlog`, the best single architecture of this project and the one carried to the 400k data.
- **Continuous-time transformer** (`ct_transformer`, 235k parameters) with learned time-difference attention biases: 0.6–0.7 MAE worse than the GRU family on one seed. Dropped.
- **Ranking loss repaired.** With `margin: -0.04` (−20 cells at the target scale of 500) the `MarginRankingLoss` fires only on pairs mis-ordered by more than 20 cells, which is 0.03 % of random pairs on a trained model, so every earlier `margin_loss_w` sweep was sweeping a dead knob. `40_gru_xl_margin_0` (margin 0) gives +0.13 points correct and −0.03 MAE, inside the confidence interval; mined hard pairs (`39`) hurt by 0.10 MAE. Margin 0 is kept as the default, hard pairs are dropped.
- `-train_on_val`: training on train+val for a fixed epoch budget. For `24_gru_xl` 12.5 % more data of the same kind gave 4.501 versus 4.457 (one seed, inside noise).

3.2 Prediction averaging and stacking

Averaging the test predictions of independently trained runs is the most reliable gain found on the 200k data (Figure 2): 4.50 for one run, 4.36 for two, 4.29 for four, 4.24 for nine, with an asymptote near 4.20 because residuals of any two members correlate at 0.82–0.93. Median and trimmed means are no better than the mean; global non-negative or ridge weights fitted on validation add 0.02; regime-gated weights by dose count (`stack_by_regime.py`) add 0.026 because the members specialise (`42` takes weight 0.49 on 11–19-dose protocols and 0.05 on 1–4-dose ones, `24_gru_xl` the reverse). The event models add slightly more ensemble diversity than a re-seed (residual correlation 0.85 versus 0.86).

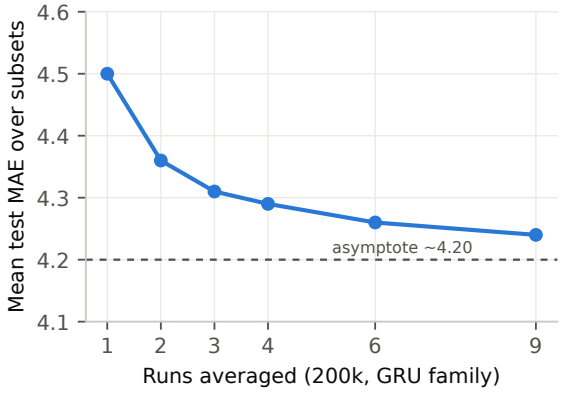


Figure 2: Mean test MAE over all subsets of k of the nine 200k GRU-family runs.

3.3 Directions closed at zero or low cost

Bigger single models; transformer variants including continuous-time ones; optimiser, scheduler and loss-shape swaps; margin sweeps at −0.04; test-time jitter of dose times (the learned function is already smooth in time); a dose-monotonicity constraint (adding 0.125 Gy lowers the prediction in

96.6–96.9 % of protocols already); median and trimmed ensembles; global stacking beyond +0.02; gradient boosting on a flat encoding (12.5 MAE, so the sequence models extract real structure).

4 Results on the original 200k data

Appendix A says what each configuration changes; Appendix B lists every finished run.

run (200k test split)	MAE	RMSE	MAPE	correct	MAE_{99}
`1.yml` baseline, 4 runs	4.52–4.59	5.98	1.08 %	94.9–95.0 %	17.5
`24_gru_xl` s1	4.457	5.83	1.055 %	95.07 %	17.2
`24_gru_xl` s2	4.521	5.96	1.070 %	95.10 %	17.5
`41_event_gru` s1	4.510	5.92	1.068 %	94.91 %	17.1
`42_event_gru_addlog` s1	4.399	5.77	1.044 %	95.13 %	16.8
`42_event_gru_addlog` s2	4.383	5.72	1.039 %	95.21 %	16.7
`42_event_gru_addlog` s3	**4.289**	**5.61**	**1.017 %**	95.29 %	**16.4**
`43_gru_xl_addlog` s1	4.440	5.82	1.052 %	95.06 %	16.8
`43_gru_xl_addlog` s2	4.500	5.92	1.067 %	95.08 %	17.0
`44_ct_transformer` s1	5.190	7.05	1.230 %	94.38 %	–
`45_ct_transformer_addlog` s1	5.064	6.73	1.200 %	94.39 %	–
`40_gru_xl_margin_0` s1	4.427	5.81	1.050 %	95.20 %	17.1
`40_gru_xl_margin_0` s2	4.550	5.95	1.079 %	95.05 %	17.4
`39_gru_xl_hardpairs` s1	4.561	6.00	1.079 %	95.06 %	17.6
`24_gru_xl` ×2	4.352	5.69	1.030 %	95.20 %	16.5
`42` ×3	4.222	5.51	1.001 %	95.36 %	16.0
all 14 GRU-family runs	**4.186**	**5.45**	**0.992 %**	**95.35 %**	**15.8**

Table 2: Single runs and prediction averages on the 200k test split. Confidence intervals are 95 % bootstrap intervals of the MAE.

Against the methods the project started from: every run in Table 2 outside the transformers beats the thesis' best model on MAPE, RMSE, MAE, correct-percent and MAE_{99} (thesis 4.87 MAE; worst 4.59, best 4.29). The best single 200k model is `42` seed 3 (4.289 / 95.29 %); the reproduced baseline `1.yml` sits at 4.55. The 14-run GRU-family average is the first result under 4.2 MAE and under 1 % MAPE and matches or edges the upstream README's headline on every column (4.186 vs 4.2 MAE, 0.992 vs 0.99 % MAPE, 95.35 vs 95.3 % correct, 15.76 vs 15.78 MAE_{99}).

5 The second 200k dataset and the 400k combination

5.1 Why more data

The learning curve on subsets of the original data (Figure 5) was still sloping: training `24_gru_xl` on 25 %, 50 % and 100 % of the 200k series gives 5.25, 4.81 and 4.46 MAE. Halving the data costs 0.35 MAE, so data volume was the one lever with a measured effect that had not been pulled.

5.2 Sampling: scrambled Sobol instead of uniform-random

The new protocols were drawn from a scrambled Sobol sequence (seed 42) in $1 + 2 \times 20$ dimensions under the constraint set of the original paper (Section 2.2): total dose in $[0.75, 10]$ Gy, each dose at least 0.5 Gy and a multiple of 0.125 Gy, dose times multiples of 100 steps in $[0, 72\,000]$ and at least 300 steps apart. One coordinate picks the number of doses; each following (time, dose) pair greedily places one dose on the still-admissible time grid and remaining dose budget. Because the sequence is low-discrepancy rather than i.i.d., the 200 000 protocols cover the (time, dose, count) space more evenly than the original uniform sampler. It is a constructive quasi-random sampler, not a uniform draw from the enumerated $\sim 10^{54}$-element protocol space the paper's C++ generator used, so the two sets have different marginals by design (Figure 3).

5.3 Simulation on Rysy

The EMT6-Ro simulator was built and run on the ICM Rysy cluster. The 200k protocols were split into 32 shards and simulated as one GPU array job, following the paper authors' own generation procedure with the Sobol protocols in place of their uniform draw. Each protocol was simulated 100 times on tumour 1 for 144 000 steps, the paper's Dataset 2 convention, and the target is the mean final live-cell count. The shard outputs were concatenated in order and reshaped into the long-format CSV (21 rows per series: 20 right-aligned dose slots left-padded with zeros, plus one target row), byte-compatible with the reference `data.csv`.

5.4 Combining without breaking the split

The training code splits train/validation/test by *series-id range* (< 0.8 max train, 0.8–0.9 validation, > 0.9 test). Appending the new block with an id offset would therefore have put every Sobol protocol into validation and test and none into training. The combination step instead validates both inputs (column set, 21 contiguous rows per series, one target row) and assigns every series a fresh id from one seeded permutation (seed 0), so each split holds a 50/50 source mix: 160k/20k/20k series per source. A sidecar `data_400k.csv.series_map.csv` maps every new id to (source, original id), which is what makes the like-for-like evaluation below possible. The combined file equals its two sources value-for-value (0 mismatches of 400 000 series).

Where the data lives. The Hugging Face dataset `zzox531/emt6ro-sobol-400k` holds the *full 400k training set*, not only the Sobol half (Table 3). Downloading `data_400k.csv` and its series map is sufficient to reproduce every 400k run in this note; the checksums of both files were verified against the local copies on 29 September 2026. The Sobol-only `dataset.csv` and the raw simulator output are kept in the same repository for traceability. The uniform 200k `data.csv` is not stored there separately: it is embedded in `data_400k.csv` (recoverable through the series map) and is otherwise regenerated by `python -m cancer_nn.preprocess_data` from the paper authors' GitHub repository.

file	series	size	content
`data_400k.csv`	400 000	259 MB	uniform 200k + Sobol 200k, shuffled ids; SHA-256 `63ed5813...`
`data_400k.csv.series_map.csv`	400 000	8 MB	new id → (source, original id); SHA-256 `5a0c4531...`
`dataset.csv`	200 000	162 MB	Sobol protocols only, training format
`protocols_all.txt`, `results_all.txt`	200 000	20 + 80 MB	raw Sobol protocols and the 100 replicate counts per protocol
`SHA256SUMS`, `SHA256SUMS_400k`, `README.md`			checksums and the dataset card

Table 3: Contents of the Hugging Face dataset `zzox531/emt6ro-sobol-400k`.

5.5 Validation that the two sources agree

A comparison of the Sobol set against the reference found identical schema, dtypes, layout, dose grid, time grid and `time_gap` convention. Target given protocol, which must agree if the simulator is the same, does: the 10-nearest-neighbour bias of Sobol targets against reference neighbours is -0.75 ± 0.61 cells versus $+0.29 \pm 0.46$ for reference-against-reference, and a gradient-boosting surrogate fitted on the reference scores 13.26 MAE on in-support Sobol protocols versus 12.51 on a reference hold-out. The marginals differ as intended (Table 4): dose count is close to uniform over 1–16 instead of peaking at 7–8, 62 % of Sobol protocols use at least 9.75 Gy of the 10 Gy budget versus 31 %, and the target spread is wider (standard deviation 55 versus 39 cells).

source	series	doses (mean)	total Gy (mean)	total $\geq$ 9.75 Gy	target mean	target std
uniform (reference)	200 000	7.84	9.08	31.1 %	426.5	38.7
Sobol	200 000	9.15	9.37	62.4 %	420.6	54.8

Table 4: Per-source protocol statistics of `data_400k.csv`, computed through the series map.

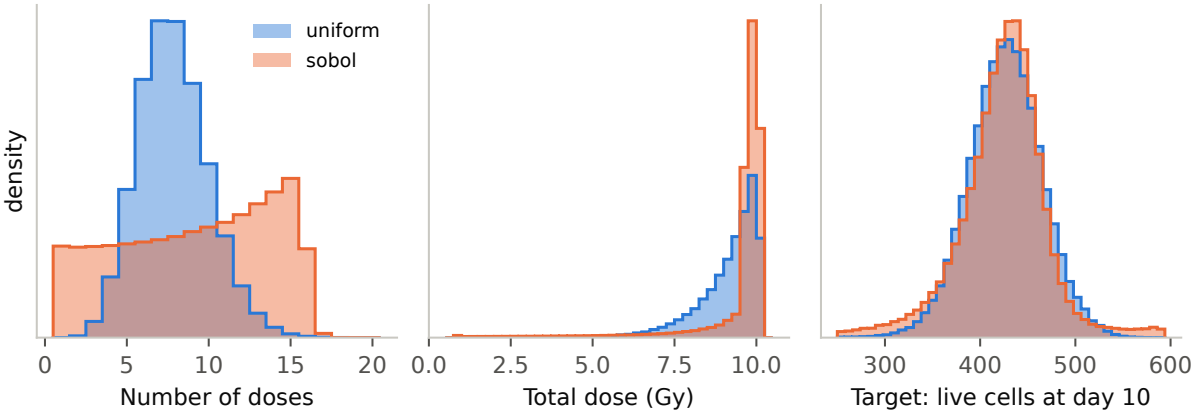


Figure 3: Marginal distributions of the two sources in the 400k set.

5.6 Consequence for evaluation

The 400k test split (40 000 series, half Sobol, wider dose range) is not the old 20k test split, so raw `test/*` numbers of 400k runs are not comparable with the 200k tables. Every 400k number quoted as like-for-like in this note is filtered to `source == uniform` through the sidecar. The two uniform test sets are then the same distribution but different protocols, so differences are judged against the bootstrap intervals.

6 What the 400k data leveraged

`run_400k_baseline.sh` reproduced the two reference configurations on `data/data_400k.csv`: `42_event_gru_addlog` seeds 1–3 with a 300-epoch cap and `24_gru_xl` seeds 1–2 with a 200-epoch cap: 11.7 / 16.3 / 13.2 hours for the three `42` runs and 2.5 hours for each `24_gru_xl` run, about 46 GPU-hours in total on an RTX 5070 laptop (1 250 steps per epoch). A reboot killed the second `42` run at epoch 203 and cost a further nine hours; the manifest made the restart skip the finished runs.

model	data	MAE	RMSE	MAPE	correct	MAE_{99}	epochs
42 s1	200k	4.399	5.77	1.044 %	95.13 %	16.8	200
42 s1	400k	4.226	5.51	1.002 %	95.65 %	15.7	286
42 s2	200k	4.383	5.72	1.039 %	95.21 %	16.7	266
42 s2	400k	4.194	5.46	0.994 %	95.70 %	15.4	265
42 s3	200k	4.289	5.61	1.017 %	95.29 %	16.4	299 (cap)
42 s3	400k	**4.175**	**5.45**	**0.990 %**	**95.72 %**	15.7	266
`24_gru_xl` s1	200k	4.457	5.83	1.055 %	95.07 %	17.2	176
`24_gru_xl` s1	400k	4.273	5.59	1.012 %	95.54 %	16.0	200 (cap)
`24_gru_xl` s2	200k	4.521	5.96	1.070 %	95.10 %	17.5	182
`24_gru_xl` s2	400k	4.287	5.61	1.016 %	95.60 %	16.4	200 (cap)
42 ×3	200k	4.222 [4.17, 4.27]	5.51	1.001 %	95.36 %	16.0	
42 ×3	400k	4.089 [4.04, 4.14]	5.32	0.970 %	95.78 %	15.0	
`24_gru_xl` ×2	200k	4.352 [4.30, 4.40]	5.69	1.030 %	95.20 %	16.5	
`24_gru_xl` ×2	400k	4.153 [4.11, 4.20]	5.42	0.985 %	95.77 %	15.5	
best 200k average (14 runs)	200k	4.186 [4.14, 4.23]	5.45	0.992 %	95.35 %	15.8	
all 5 runs	**400k**	**4.033 [3.99, 4.08]**	**5.25**	**0.957 %**	**95.87 %**	**14.9**	

Table 5: Like-for-like comparison: 200k models on the 200k test split, 400k models on the uniform half of the 400k test split. Brackets are 95 % bootstrap intervals of the MAE.

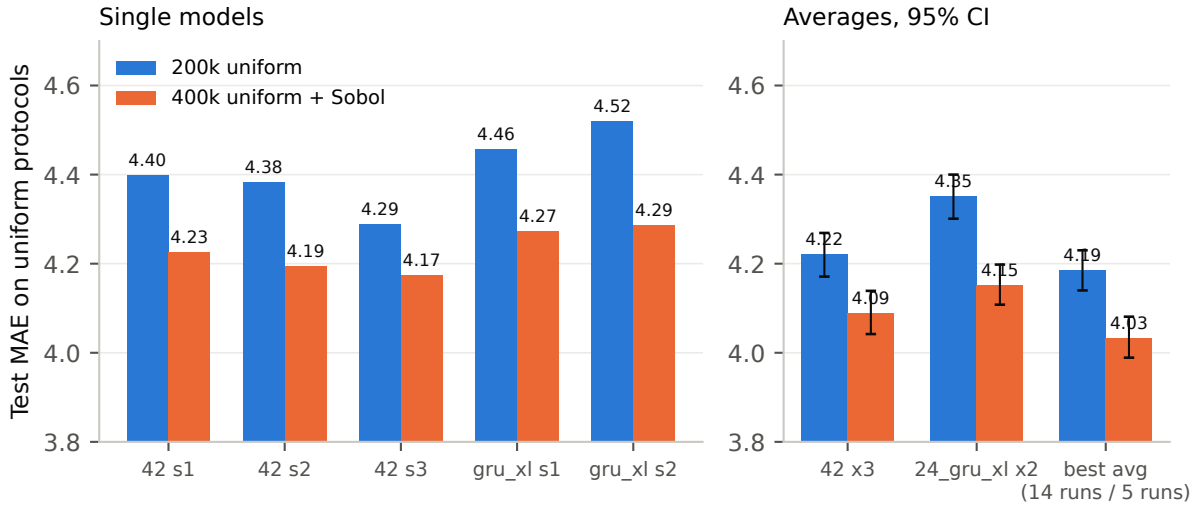


Figure 4: Test MAE on uniform protocols, 200k versus 400k training data, for the five reproduced runs and their averages.

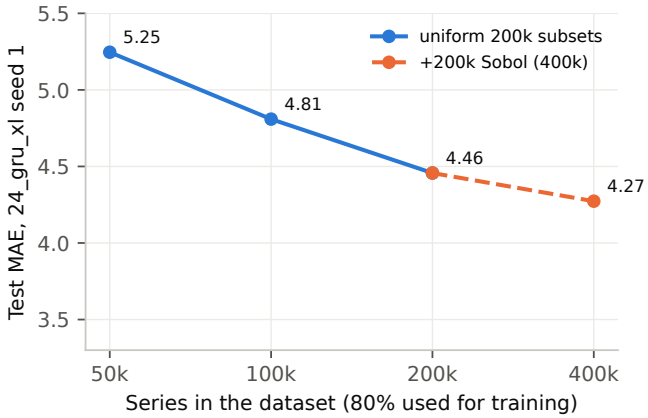


Figure 5: Learning curve of `24_gru_xl` seed 1. The 50k and 100k points are subsets of the uniform data; the 400k point is the uniform-only score of the model trained on the combined set.

What it bought.

- **Every 400k run beats its 200k twin on every metric**: MAE drops by 0.11–0.23 per run, MAE_{99} by about one cell, correct-percent rises by 0.4–0.6 points. For both group rows the 400k confidence interval lies entirely below the 200k one, so this is not seed noise.
- **Five 400k runs averaged beat the best 200k result**, the 14-run average, by 0.15 MAE and 0.5 points of correct-percent. Two single 400k models (`42` s2 and s3) are under 1 % MAPE on their own, which no 200k model achieved.
- **The learning curve has not flattened.** 100k → 200k gave −0.35 MAE, 200k → 400k gave −0.18 for the same configuration, so the gains have not reached a plateau.
- **Data mattered more than architecture.** The gap between `42` and `24_gru_xl` narrowed from about 0.10 to about 0.06 MAE, less than the 0.18 the extra data gave either of them.
- **The Sobol half is modelled as well as the uniform half.** On the full 400k split the five-run average scores 4.008 MAE / 95.86 % correct; Sobol protocols have slightly lower MAE (3.98 vs 4.03) and slightly higher MAPE because their targets are smaller on average. The model generalises to the wider dose range (single doses of 8.5–10 Gy the reference never contained) without a separate regime.
- **Both `24_gru_xl` runs hit the 200-epoch cap** with validation loss still improving slowly, whereas on 200k they early-stopped around epoch 180. More data supports longer training; a 300-epoch cap is the obvious next run.

7 Reproducing the numbers

- Environment: `conda activate cancer-nn-5070`, with `WANDB_API_KEY` exported.
- Data: `data/data_400k.csv` and its `.series_map.csv` sidecar from the Hugging Face repository. `python -m cancer_nn.preprocess_data` regenerates the original `data/data.csv`.
- 400k batch: `scripts/run_400k_baseline.sh` (resumable). `EVAL_ONLY=1` rebuilds only the table `logs/400k_baseline_table.log` with the per-source breakdown; predictions are saved in `logs/400k_baseline_preds.npz`.
- 200k tables: `STAGES="" scripts/run_next_batch.sh`, giving `logs/final_table_2026_09_15.log` and `logs/next_batch_preds.npz`.
- Learning curve: `python make_data_subsets.py`, then `scripts/run_learning_curve.sh`.
- Dataset generation is not part of this directory: `data/data_400k.csv` and its series map are downloaded from the Hugging Face repository (Table 3), which also keeps the Sobol-only file and the raw simulator output.
- Appendix B is generated from a local export of the W&B project; that export and the batch manifests carry run identifiers and are not part of the repository.

8 Open directions

In order of expected value per GPU-hour: (1) `24_gru_xl` and `42` on 400k with a 300–400-epoch cap and slower LR decay, since both were still improving; (2) more 400k seeds for averaging, which on 200k was worth 0.26 MAE from one to nine runs; (3) regime-gated stacking of the 400k members and distillation into one student; (4) a third 200k Sobol block, since the learning curve has not flattened; (5) obtaining the 256 protocols × 16 simulator re-runs behind the thesis' Table 5.2 to score against a lower-noise reference, which would make 0.05 MAE differences visible; (6) evolving the event models to the 144 000-step end of the simulation instead of the 72 000-step end of dosing, which would match the simulator's structure exactly and has not been tried.

A What each configuration changes, and why

How a configuration is read. Every file in `configs/cnn_lstm_multi_loss/` builds the same wrapper, `MultiHeadTaskRegressor`. It instantiates one sub-network per entry of `config_list` (each entry is a file in `configs/single/` holding that sub-network's hyper-parameters), feeds all of them the same series and averages their scalar outputs. The training loss is a weighted sum (`w_losses`) of a regression loss on the averaged output (`main_loss`) and one per member (`losses`), plus a `MarginRankingLoss` between this series and a second, randomly paired series from the batch, weighted by `margin_loss_w`. That ranking term is $\max\left(0,\ -s\,(\hat{y}_1 - \hat{y}_2) + m\right)$ with $s = \pm 1$ the true order. Targets are divided by 500, so the inherited margin $m = -0.04$ is -20 cells: the term is zero unless the pair is mis-ordered by more than 20 cells, which a trained model almost never does, so until config 40 the ranking loss was effectively switched off. Unless a row says otherwise the loss block is that of `1.yml` (L1 everywhere, ranking weight 5, margin -0.04), and training is Adam at 10^{-3}, batch 256, `ReduceLROnPlateau` (factor 0.7, patience 5), early stopping on validation loss with patience 20, and a 200-epoch cap. Parameter counts were obtained by instantiating each configuration. Results are test MAE / correct-percent on the 200k split, one seed unless noted.

The inherited model (`1.yml`, 392k parameters). `CNNAttLSTM` reads the 20 rows (time, dose, gap since the previous dose) in three ways at once. A bidirectional LSTM runs over the rows; a one-dimensional U-Net (`CNNContext`) convolves over the whole series and compresses it into a single query vector; two attention passes combine the two, first self-attention among the LSTM outputs, then cross-attention of the U-Net query against them. A linear layer on the attended vector gives the prediction. Three such sub-networks of 32 hidden units and 3 layers are averaged. It scores 4.52–4.59 MAE / 95.0 % over four complete runs.

19–26: a bare GRU, and how big it has to be. A GRU is a recurrent cell with two gates (update and reset) that reads the rows in order and carries a hidden state forward; the prediction is a linear function of the state after the last row. Configs 19–26 use `GRURegressor`, which is that and nothing else: no U-Net, no attention. They test whether the baseline's accuracy comes from the recurrence or from the extra branches, and how large the recurrence has to be. Four knobs are swept. *Hidden size* is the width of the state vector, i.e. how much about the protocol so far the model can carry. *Layers* stack GRUs so that the second reads the first's output sequence, which lets the model form features of features, such as a dose's effect conditional on what preceded it. *Bidirectional* adds a second GRU that reads the series backwards and concatenates the two final states, so every position also sees the doses still to come; for a protocol that is fixed in advance this is legitimate information. *Dropout* between layers is the only regularisation.

config	size	params	result	note
`19_gru_single`	16 hidden, 1 layer	1k	7.46 / 92.1 %	starting point
`20_gru_tiny`	8 hidden, 1 layer	0.3k	7.95 / 91.5 %	capacity is the bottleneck
`21_gru_medium`	32 hidden, 2 layers, dropout 0.1	10k	5.81 / 93.8 %	depth is the biggest single step
`22_gru_bidir`	16 hidden, 1 layer, bidirectional	2k	7.01 / 92.2 %	bidirectionality alone, at the size of 19
`23_gru_large`	64 hidden, 2 layers, bidir., dropout 0.2	101k	4.63 / 4.65 / 95.0 % (s1, s2)	first GRU in the baseline band
`24_gru_xl`	96 hidden, 3 layers, bidir., dropout 0.2	392k	4.457 / 4.521 / 95.1 % (s1, s2)	reference single model; base of 27–43
`25_gru_xxl_single`	128 hidden, 4 layers, bidir., dropout 0.25	992k	4.62 / 4.51 / 95.0 % (s1, s2)	no better than 24
`26_gru_ensemble_diverse`	medium + large + xl averaged	503k	4.46 / 4.59 / 95.1 % (s1, s2)	within noise of 24

Depth matters most (21 against 19), bidirectionality helps a little on its own (22 against 19), and the error falls monotonically up to 24 and then stops: 25 has 2.5 times the parameters of 24 and is no better. 24 therefore became the reference single model. It has the same parameter count as

`1.yml` and beats it, so the U-Net and attention branches were not what made the baseline work. 26 applies the upstream recipe of averaging three members of different size inside one model, each with its own L1 head; it is within seed noise of 24 alone. A second seed of 23–26 followed by Welch's t-test found the four indistinguishable, so the sweep stopped at xl.

27–29: the U-Net and attention branches put back on a GRU. `CNNAttGRU` is `CNNAttLSTM` with its LSTM replaced by a GRU of exactly the size used in 21, 23 and 24, so each pair of configs differs only by the presence of the U-Net query and the two attention passes. If those branches carry information the recurrence misses, the member with them should win at every size.

config	same core as	params	result	against the plain GRU
`27_cnn_att_gru_medium`	21 (32/2)	44k	5.72 / 93.7 %	better (5.81)
`28_cnn_att_gru_large`	23 (64/2 bidir.)	295k	4.57 / 95.0 %	better (4.63)
`29_cnn_att_gru_xl`	24 (96/3 bidir.)	775k	4.565 / 95.0 %	worse (4.457) at twice the parameters

The branches help a core that is too small to summarise the protocol on its own and become redundant, then harmful, once the core is large enough. The wrapper was not carried forward.

37–38: optimiser, scheduler, loss shape and weight averaging. Four training-process changes that are standard elsewhere, stacked on the two best models. *Huber loss* is quadratic for residuals below a threshold and linear above it, so it has smooth gradients near zero without the outlier sensitivity of a squared loss. *AdamW* applies weight decay to the weights directly instead of through the gradient (decoupled L2 regularisation, here 10^{-4}). *OneCycle* replaces the plateau-driven decay with a fixed schedule that ramps the learning rate up and then anneals it to zero over the epoch budget. *Stochastic weight averaging* (`-swa_lrs 0.0005`) keeps a running average of the weights over the final epochs, which tends to settle in a flatter minimum.

config	base	params	result	against the plain run
`37_lstm_best_twitched`	`1.yml` + the four changes	392k	4.885 / 94.6 %	worse (4.55)
`38_gru_best_twitched`	`24_gru_xl` + the four changes	392k	4.856 / 94.9 %	worse (4.46)

Both are clearly worse than their plain counterparts. The likely reasons are the schedule, which commits to the full epoch budget where the plain runs stop at the validation optimum, and the loss, which is quadratic exactly where this problem's residuals live (a few cells on a target of several hundred) and so flattens the gradient there. The stack was dropped; the four knobs remain in the code as options.

39–40: making the ranking loss do something. Margin 0 makes the ranking term penalise every mis-ordered pair in proportion to how far the two predictions are from the right order. Config 40 is 24 with only that change and serves as the control. Config 39 adds *hard-pair mining*: instead of pairing each series with a uniformly random partner, half of the partners are drawn from series whose true target lies 4 to 15 cells away, pairs that are difficult to order but not so close that noise in the labels decides the order.

config	change from 24	params	result	verdict
`40_gru_xl_margin_0`	margin 0, random pairs	392k	4.427 / 4.550 / 95.2 % (s1, s2)	+0.13 pt correct, −0.03 MAE, inside the CI; kept
`39_gru_xl_hardpairs`	margin 0 + hard pairs	392k	4.561 / 95.1 %	0.10 MAE worse than 40; dropped

Margin 0 gives a small gain in ordering accuracy and MAE that sits inside the bootstrap interval, so it is the recommended setting but not a result. Hard pairs hurt: forcing the model to order near-ties spends capacity on distinctions the labels themselves cannot support.

41–45: models shaped like the simulator. The simulator behind the data evolves a tumour continuously and applies doses at instants. A GRU stepping over a fixed 20-row grid with zero padding is a poor match for that. `EventRecurrentRegressor` (`mode: event_gru`, configs 41–42) mirrors the simulator's structure. A hidden state starts from a learned initial vector. Between two doses it is *evolved* by a residual MLP that takes the state and the elapsed gap on a linear and a log scale, applied in `flow_steps` sub-steps; this is where repopulation and repair can be represented. At each dose a `GRUCell` applies a *jump* conditioned on the dose, its square and its time; this is where cell kill can be represented. After the last dose the state is evolved to the end of the 72 000-step dosing window (the simulation itself continues to 144 000 steps; the five untreated days are left to the readout), and padding rows never touch the state, so the number of doses does not matter. The `additive_log` readout (configs 42, 43, 45) changes only the output head: the prediction becomes $\exp\left(\log y_0 + \sum_i c(h_i) + c_{\text{end}}(h_T)\right)$, leftover cells as the mean target y_0 times one multiplicative factor per dose and one for the end of treatment, which is the form radiobiology suggests. The factor heads are initialised at zero, so the model starts out predicting the mean. Config 43 adds only the readout to 24 so that the two ideas can be separated. `ContinuousTimeTransformer` (`mode: ct_transformer`, configs 44–45) is the attention-based version of the same idea: the tokens are the doses only, each carrying dose, dose squared, absolute time, gap from the previous dose and time left to the horizon, and every attention head adds a learned bias computed from the signed time difference between two doses, so "how long ago" is explicit rather than inferred from a position index. The prediction is a masked mean over the dose tokens, or the additive log readout.

config	change	params	result (200k)	note
`41_event_gru`	event recurrence, linear readout (96 hidden, flow MLP 192, 2 sub-steps)	94k	4.510 / 94.9 %	slightly worse than 24
`42_event_gru_addlog`	41 + additive log readout	99k	4.399 / 4.383 / 4.289 / 95.1–95.3 % (s1–s3; s2, s3 at a 300-epoch cap)	best single model; 4.23 / 4.19 / 4.18 on 400k (uniform test protocols)
`43_gru_xl_addlog`	24 + additive log readout only	411k	4.440 / 4.500 / 95.1 % (s1, s2)	readout alone ≈ 0.02 MAE, within noise
`44_ct_transformer`	event transformer, linear readout (d_model 96, 4 heads, 3 layers)	235k	5.190 / 94.4 %	dropped
`45_ct_transformer_addlog`	44 + additive log readout	240k	5.064 / 94.4 %	dropped

Event recurrence alone is slightly worse than the plain GRU and the readout alone is within noise, but together (42) they give the best single model on 200k and, repeated on 400k, 4.18–4.23 MAE. The combination works because the readout asks for exactly the per-dose quantity the event recurrence produces. The continuous-time transformer is 0.6–0.7 MAE behind on one seed and was dropped: with at most 20 events per protocol there is little for attention to learn that the recurrence does not already capture.

B Every finished run

Table 6: Every finished run in the W&B project of the baseline reproduction and of configs 19 onwards, by config. Rows marked 400k were trained on `data_400k.csv` and are scored on its test split (not comparable with 200k rows; see Section 6 for the uniform-only numbers).

config	seed	ep	MAE	RMSE	MAPE	correct	MAE_{99}	note
`1`	1	170	4.549	5.96	1.08%	94.91%	17.6	
	1	200	4.561	5.99	1.08%	94.97%	17.5	
	1	197	4.589	6.03	1.09%	94.95%	17.6	
	2	200	4.521	5.92	1.07%	95.04%	17.3	
`19_gru_single`	1	200	7.462	10.03	1.77%	92.08%	30.8	
`20_gru_tiny`	1	200	7.955	10.68	1.89%	91.46%	32.9	

config	seed	ep	MAE	RMSE	MAPE	correct	MAE_{99}	note
21_gru_medium	1	169	5.812	7.75	1.38%	93.79%	23.5	
22_gru_bidir	1	200	7.009	9.42	1.66%	92.24%	29.0	
23_gru_large	1	200	4.631	6.06	1.10%	95.06%	17.5	
	2	200	4.645	6.11	1.10%	94.95%	17.9	
24_gru_xl	1	176	4.457	5.83	1.06%	95.07%	17.2	
	2	182	4.521	5.96	1.07%	95.10%	17.5	
	1	192	5.246	6.95	1.24%	93.84%	21.3	
	1	155	4.501	5.91	1.07%	95.15%	17.1	
	1	148	4.809	6.34	1.14%	94.86%	18.7	
	1	200	4.266	5.53	1.03%	95.59%	15.5	400k
	2	200	4.267	5.54	1.03%	95.60%	15.9	400k
25_gru_xxl_single	1	136	4.624	6.07	1.09%	94.98%	17.9	
	2	144	4.511	5.92	1.07%	95.03%	17.4	
26_gru_ensemble_diverse	1	177	4.463	5.82	1.06%	95.13%	17.0	
	2	155	4.586	6.02	1.09%	95.00%	17.6	
27_cnn_att_gru_medium	1	94	5.715	7.62	1.37%	93.70%	23.1	
28_cnn_att_gru_large	1	164	4.570	5.98	1.08%	94.99%	17.5	
29_cnn_att_gru_xl	1	147	4.565	5.98	1.08%	94.95%	17.3	
37_lstm_best_twitched	1	185	4.885	6.40	1.16%	94.63%	18.7	
38_gru_best_twitched	1	170	4.856	6.39	1.15%	94.85%	18.9	
39_gru_xl_hardpairs	1	164	4.561	6.00	1.08%	95.06%	17.6	
40_gru_xl_margin_0	1	157	4.427	5.81	1.05%	95.20%	17.1	
	2	105	4.550	5.95	1.08%	95.05%	17.4	
41_event_gru	1	200	4.510	5.92	1.07%	94.91%	17.1	
42_event_gru_addlog	1	200	4.399	5.77	1.04%	95.13%	16.8	
	2	266	4.383	5.72	1.04%	95.21%	16.7	
	3	299	4.289	5.61	1.02%	95.29%	16.4	
	1	286	4.188	5.44	1.01%	95.74%	15.4	400k
	2	265	4.165	5.40	1.01%	95.72%	15.3	400k
	3	266	4.159	5.40	1.01%	95.68%	15.4	400k
43_gru_xl_addlog	1	200	4.440	5.82	1.05%	95.06%	16.8	
	2	175	4.500	5.92	1.07%	95.08%	17.0	
44_ct_transformer	1	185	5.190	7.05	1.23%	94.38%	21.0	
45_ct_transformer_addlog	1	200	5.064	6.73	1.20%	94.39%	20.2	